

# DEVELOPMENT OF A DATA MONITORING TOOL FOR THE LEMING EXPERIMENT

-

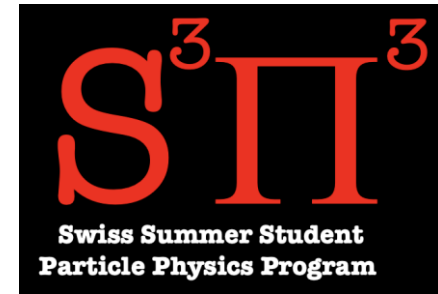
## UPDATED LEMING BROWSER

PROGRESS MEETING #1

AUGUST 6TH, 2026



# Agenda



## 01 Evolution of the Data Architecture

Legacy structure → New architecture

## 02 The New LEMING Browser

Architecture, robustness, maintainability & features

## 03 Software Architecture

Design principles, classes & implementation

## 04 Usage Demonstration

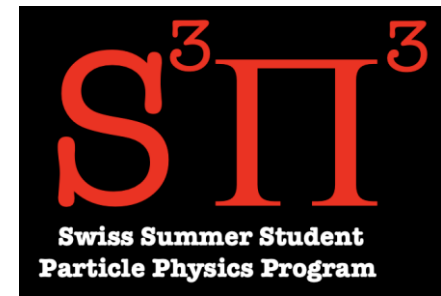
Typical workflows and examples

## 05 Results & Future Work

Performance, conclusions and outlook



# Legacy LEMING Browser



## Main Capabilities

- ROOT-based graphical interface
- Interactive histogram visualization
- Detector-specific cuts & corrections
- Comparison of two datasets
- Configuration save/load

## Workflow

ROOT File → Select Run(s) → Select Histograms → Draw Histograms

LEMING Browser

Welcome to the LEMING Browser!

Enter the path to where your data is stored below, e.g. ~path/to/my/files. If no path is specified, the default path (data/leming/ST2025/leaf or data/leming/ST2025/beam for silicon strips) is chosen.

Runs can be a comma-separated list of run numbers (e.g. 02611, 0001, 02611, 0003) and run ranges (e.g. 02611-02677 or 02511 (all subruns of this run merged together)) or if you'd like to call a single file, give its full name (e.g. 02116\_00000 or run02116\_00000.mid.root)

The dataset name which should be used in the legend of the histograms can be chosen. If no name is specified, the run range is used.

Enter dataset 1

Enter path:  Browse Folders Enter runs:  Enter name:  Use E cuts of Cutdown number:

Enter dataset 2 (optional)

Enter path:  Browse Folders Enter runs:  Enter name:  Use E cuts of Cutdown number:

Select histogram

☐ Lifetime corrected

☐ Background subtracted

☐ Degraded

☐ with E cuts

☐ atomic colic + atomic E cut

☐ atomic colic + atomic time cut

☐ Choose volume of interest (only for Silicon strip histogram) - default: ('x': (-6.0, 6.0), 'y': (17.0, 29.0), 'z': (10.0, 30.0))

Enter x range of volume of interest (in mm, e.g. -6-6):

Enter y range of volume of interest (in mm, e.g. 17-29):

Enter z range of volume of interest (in mm, e.g. 10-30):

Select configuration

☐ Use own config file. Config file name:

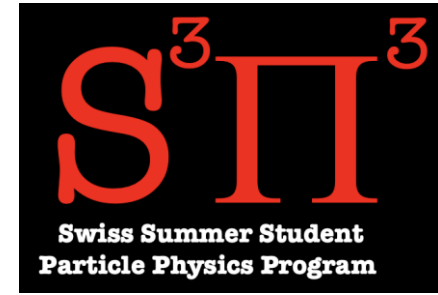
☐ Use default config file

Draw selected histograms

Name of new config file:  Save Config Exit



# Challenges of the Legacy Implementation



## Architecture

- Monolithic implementation
- GUI tightly coupled with data processing
- Histogram logic embedded in GUI
- Growing number of special-case implementations

## Maintainability

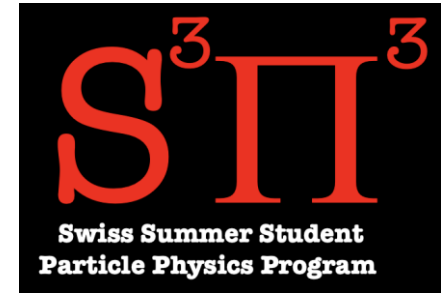
- Difficult to extend for new detectors
- Slow performance due to the need to load histogram files before analysis.
- File-loading overhead significantly increased startup and browsing times.
- No multi-threading support
- Hardcoded energy & time cuts
- Multiple files modified for new features
- Increasing code complexity

## Motivation

A complete redesign was required to support the new BT26 data architecture while improving modularity, maintainability and extensibility.



# Legacy ROOT File Structure



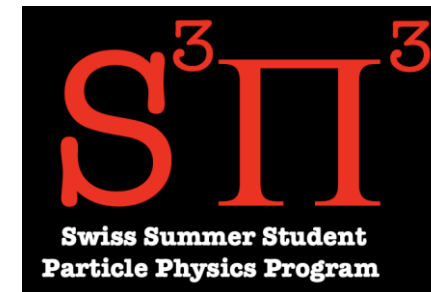
```
(leming-edge) kfritzas@lepp-obi-wan: /run/user/1032/gvfs/sftp:host=lepp-c-3po,user=leming/data0/leming/BT2025/loaf$ root run03451_00000.mid.root
root [0]
Attaching file run03451_00000.mid.root as _file0...
(TFile *) 0x5db9d7e18900
root [1] .ls
TFile**
TFile*
KEY: TH1F      test_strip;1      test_strip
KEY: TH2F      hElectronEnergy;1      Electron detector energy
KEY: TH2F      hMuonEnergy;1      Muon detector energy
KEY: TH2F      hAtomicEnergy;1      Atomic detector energy
KEY: TH1F      hCoincidencePP_L_F1aN1;1      Electron detector coincidence versus time PP L F1aN1
KEY: TH1F      hCoincidencePP_L_F1aN2;1      Electron detector coincidence versus time PP L F1aN2
KEY: TH1F      hCoincidencePP_L_F2aN1;1      Electron detector coincidence versus time PP L F2aN1
KEY: TH1F      hCoincidencePP_L_F2aN2;1      Electron detector coincidence versus time PP L F2aN2
KEY: TH1F      hCoincidencePP_L_F2aN3;1      Electron detector coincidence versus time PP L F2aN3
KEY: TH1F      hCoincidencePP_L_F3aN2;1      Electron detector coincidence versus time PP L F3aN2
KEY: TH1F      hCoincidencePP_L_F3aN3;1      Electron detector coincidence versus time PP L F3aN3
KEY: TH1F      hCoincidencePP_R_F1aN1;1      Electron detector coincidence versus time PP R F1aN1
KEY: TH1F      hCoincidencePP_R_F1aN2;1      Electron detector coincidence versus time PP R F1aN2
KEY: TH1F      hCoincidencePP_R_F2aN1;1      Electron detector coincidence versus time PP R F2aN1
KEY: TH1F      hCoincidencePP_R_F2aN2;1      Electron detector coincidence versus time PP R F2aN2
KEY: TH1F      hCoincidencePP_R_F2aN3;1      Electron detector coincidence versus time PP R F2aN3
KEY: TH1F      hCoincidencePP_R_F3aN2;1      Electron detector coincidence versus time PP R F3aN2
KEY: TH1F      hCoincidencePP_R_F3aN3;1      Electron detector coincidence versus time PP R F3aN3
KEY: TH1F      hCoincidencePP_L_F1aN1_atomic;1      Atomic and Electron detector coincidence versus time PP L F1aN1
KEY: TH1F      hCoincidencePP_L_F1aN2_atomic;1      Atomic and Electron detector coincidence versus time PP L F1aN2
KEY: TH1F      hCoincidencePP_L_F2aN1_atomic;1      Atomic and Electron detector coincidence versus time PP L F2aN1
KEY: TH1F      hCoincidencePP_L_F2aN2_atomic;1      Atomic and Electron detector coincidence versus time PP L F2aN2
KEY: TH1F      hCoincidencePP_L_F2aN3_atomic;1      Atomic and Electron detector coincidence versus time PP L F2aN3
KEY: TH1F      hCoincidencePP_L_F3aN2_atomic;1      Atomic and Electron detector coincidence versus time PP L F3aN2
KEY: TH1F      hCoincidencePP_L_F3aN3_atomic;1      Atomic and Electron detector coincidence versus time PP L F3aN3
KEY: TH1F      hCoincidencePP_R_F1aN1_atomic;1      Atomic and Electron detector coincidence versus time PP R F1aN1
KEY: TH1F      hCoincidencePP_R_F1aN2_atomic;1      Atomic and Electron detector coincidence versus time PP R F1aN2
KEY: TH1F      hCoincidencePP_R_F2aN1_atomic;1      Atomic and Electron detector coincidence versus time PP R F2aN1
KEY: TH1F      hCoincidencePP_R_F2aN2_atomic;1      Atomic and Electron detector coincidence versus time PP R F2aN2
KEY: TH1F      hCoincidencePP_R_F2aN3_atomic;1      Atomic and Electron detector coincidence versus time PP R F2aN3
KEY: TH1F      hCoincidencePP_R_F3aN2_atomic;1      Atomic and Electron detector coincidence versus time PP R F3aN2
KEY: TH1F      hCoincidencePP_R_F3aN3_atomic;1      Atomic and Electron detector coincidence versus time PP R F3aN3
KEY: TH1F      hCoincidencePP_C61aC62;1      Crate coincidence versus time PP C61aC62
KEY: TH1F      hCoincidencePP_C62aC63;1      Crate coincidence versus time PP C62aC63
KEY: TH1F      hCoincidencePP_C63aC64;1      Crate coincidence versus time PP C63aC64
KEY: TH1F      hCoincidencePP_C64aC65;1      Crate coincidence versus time PP C64aC65
KEY: TH1F      hCoincidencePP_C65aC61;1      Crate coincidence versus time PP C65aC61
KEY: TTree      t2;1      t2
KEY: TTree      tCoincidencePP;1      tCoincidencePP
KEY: TTree      t2_tram;1      t2_tram
```

## Highlights

- Precomputed TH1/TH2 histograms stored directly in the ROOT file
- Small number of TTrees with analysis-specific content



# New ROOT File Structure



```

root [0]
Attaching file run03541_00000.mid.root as _file0...
(TFile *) 0x6460c1504bb0
root [1] .ls
TFile** run03541_00000.mid.root
TFile* run03541_00000.mid.root
KEY: TTree t2_tram;1 t2_tram
KEY: TTree t2;1 t2
KEY: TTree diagnostics;1 diagnostics
KEY: TTree rates;1 rates
KEY: TTree temperatures;1 temperatures
root [2] t2_tram->Scan()
=====
Row * IEventNb * Itime.Iti * Iedeposit * Idetname * IchamberN * IDriftTim * Runnumber * Subrunnum *
=====
0 * 1 * 0 * 1940.7921 * C61 * 0 * 0 * 3541 * 0 *
1 * 1 * 0 * 1939.2475 * C62 * 0 * 0 * 3541 * 0 *
2 * 1 * 0 * 2031.4315 * C63 * 0 * 0 * 3541 * 0 *
3 * 3 * 957.5 * 246.80917 * RF3 * 0 * -2.5 * 3541 * 0 *
4 * 3 * 1335 * 37.207374 * Atomic * 0 * 375 * 3541 * 0 *
5 * 3 * 960 * 355.77584 * TriggerSc * 0 * 0 * 3541 * 0 *
6 * 3 * 5 * 2428.6997 * NIM Mu * 0 * -955 * 3541 * 0 *
7 * 3 * 0 * 1426.9631 * Muon Entr * 0 * -960 * 3541 * 0 *
8 * 3 * 960 * 167.26184 * Strip_H2L * 24 * 0 * 3541 * 0 *
9 * 3 * 960 * 459.60351 * Strip_H1R * 18 * 0 * 3541 * 0 *
10 * 3 * 960 * 389.94955 * Strip_H2R * 6 * 0 * 3541 * 0 *
11 * 3 * 960 * 168.24635 * Strip_V1R * 9 * 0 * 3541 * 0 *
12 * 3 * 960 * 234.56962 * Strip_V1R * 12 * 0 * 3541 * 0 *
13 * 3 * 960 * 281.54699 * Strip_V2R * 18 * 0 * 3541 * 0 *
14 * 4 * 6457.5 * 33.207374 * Atomic * 0 * -27.5 * 3541 * 0 *
15 * 4 * 6485 * 274.23782 * TriggerSc * 0 * 0 * 3541 * 0 *
16 * 4 * 0 * 2427.6997 * NIM Mu * 0 * -6485 * 3541 * 0 *
17 * 4 * 0 * 2259.9631 * Muon Entr * 0 * -6485 * 3541 * 0 *
18 * 4 * 6485 * 220.63037 * Strip_V2L * 6 * 0 * 3541 * 0 *
19 * 4 * 6485 * 196.67735 * Strip_H1L * 27 * 0 * 3541 * 0 *
20 * 4 * 6485 * 249.41662 * Strip_H2L * 27 * 0 * 3541 * 0 *
21 * 5 * 4410 * 198.23782 * TriggerSc * 0 * 0 * 3541 * 0 *
22 * 5 * 2.5 * 2428.6997 * NIM Mu * 0 * -4407.5 * 3541 * 0 *
23 * 5 * 0 * 1377.9631 * Muon Entr * 0 * -4410 * 3541 * 0 *
24 * 5 * 4410 * 281.84143 * Strip_V1L * 9 * 0 * 3541 * 0 *
Type <CR> to continue or q to quit ==> q
(long long) 25
root [3] diagnostics->Scan()
=====
Row * Instance * pileupTyp * hasPileup * hasMuonEn * isTimingE * tc61vsC62 * tc63vsC62 *
=====
0 * 0 * 0 * 0 * 0 * 1 * 0 * 0 *
1 * 0 * 1 * 0 * 0 * 0 * nan * nan *
2 * 0 * 0 * 0 * 1 * 0 * nan * nan *
3 * 0 * 0 * 0 * 1 * 0 * nan * nan *
4 * 0 * 0 * 0 * 1 * 0 * nan * nan *
5 * 0 * 2 * 1 * 0 * 0 * nan * nan *
6 * 0 * 1 * 1 * 0 * 0 * nan * nan *
7 * 0 * 2 * 1 * 0 * 0 * nan * nan *
8 * 0 * 1 * 1 * 0 * 0 * nan * nan *
9 * 0 * 0 * 0 * 1 * 0 * nan * nan *

```

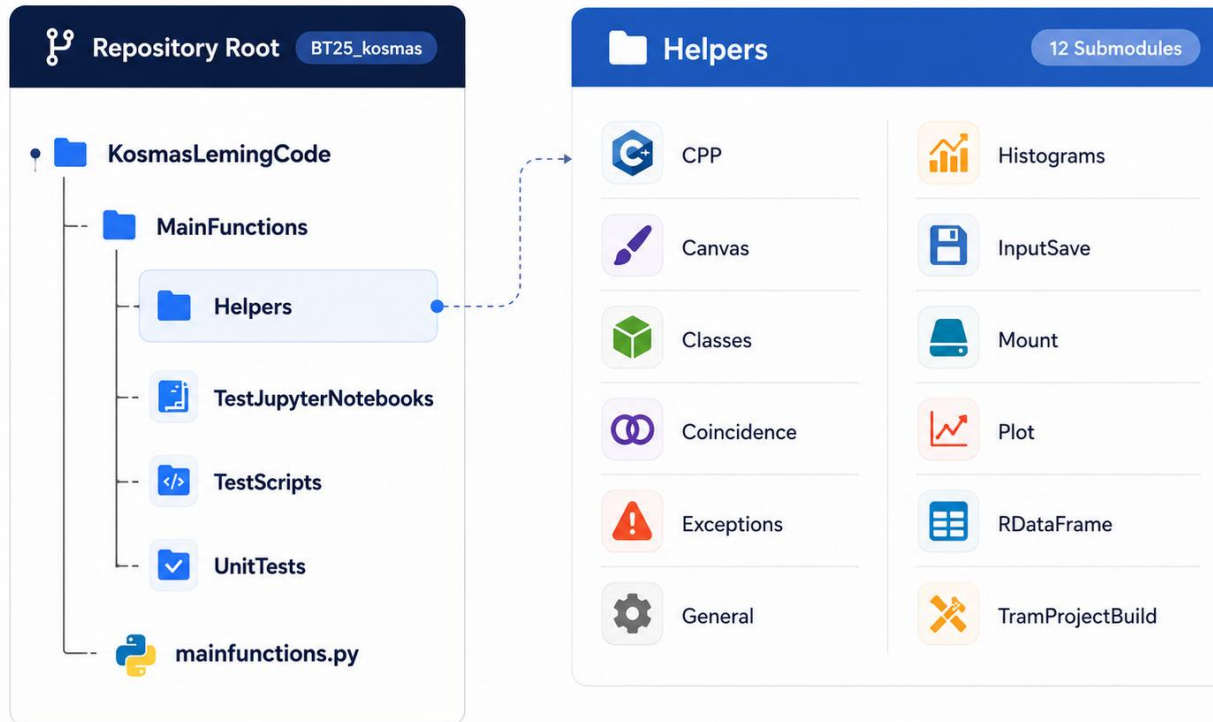
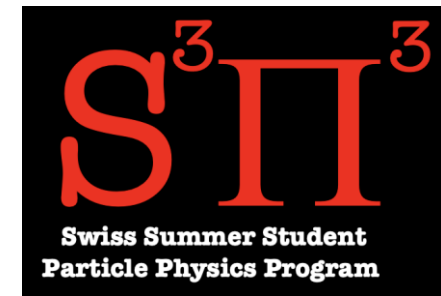
## Highlights

- Event-based TTrees store raw detector information
- Histograms are generated dynamically during processing
- Naturally extensible to additional detectors and observables
- Designed for modular, multi-threaded RDataFrame workflows





# New LEMING Browser – Code Architecture



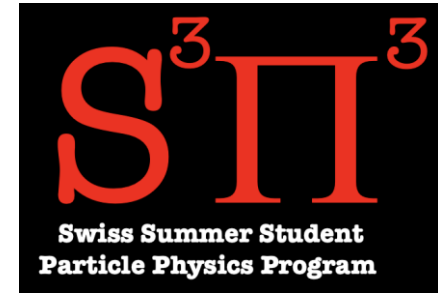
## New Implementation Improvements

- Multi-threaded processing (work no longer runs in the main thread)
- No GUI dependency — data-processing code can evolve independently
- Modular architecture enables special-case implementations without increasing complexity
- Easily extensible to support new detector types
- Native support for processing multiple datasets
- Processing logic separated into dedicated Python helper modules
- Cleaner, simpler and more maintainable codebase

**These architectural improvements are explained in the following slides.**



# C++ Helper Layer



## Why C++ Helpers?

- Core data operations run in ROOT C++, because Python functions (e.g., filtering, grouping, or accessing the first element with `get(0)`) cannot be executed efficiently in a multi-threaded `RDataFrame` loop.
- Enable efficient multi-threaded execution through ROOT's C++ backend.
- Python is used for the overall workflow and setup, while the event processing is done in ROOT's multi-threaded C++ backend.

## Implemented Helpers

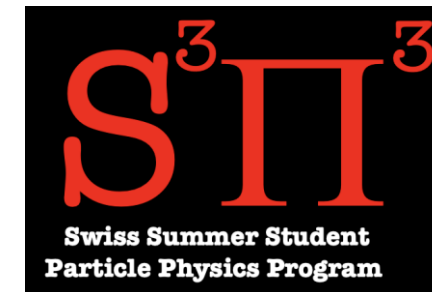
- Detector identification (`GetDetectorName`)
- Run/Subrun extraction from filenames
- Smart vector filtering for `RVec` columns
- Mask broadcasting and validation

Result: Fast, reusable C++ primitives that seamlessly integrate with Python and `RDataFrame`.





# RDataFrame Helper



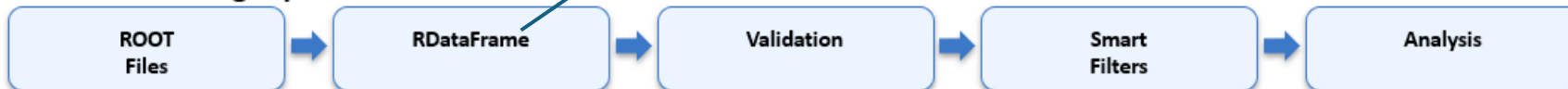
## Main Responsibilities

- Resolve run specifications into ROOT files
- Configure ROOT thread pool
- Add Run/Subrun metadata from filenames
- Apply smart scalar and RVec cuts
- Create derived columns for straight coincidences

```
if number_of_threads is None:
    ROOT.EnableImplicitMT()
else:
    if number_of_threads < 1:
        try:
            raise DataFrameError("number_of_threads must be at least 1")
        except DataFrameError as err:
            print(err)
            sys.exit(1)
    ROOT.EnableImplicitMT(number_of_threads)

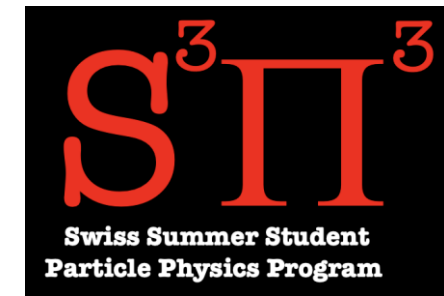
print(f"\nROOT threads: {ROOT.GetThreadPoolSize()}")
```

### Data Processing Pipeline



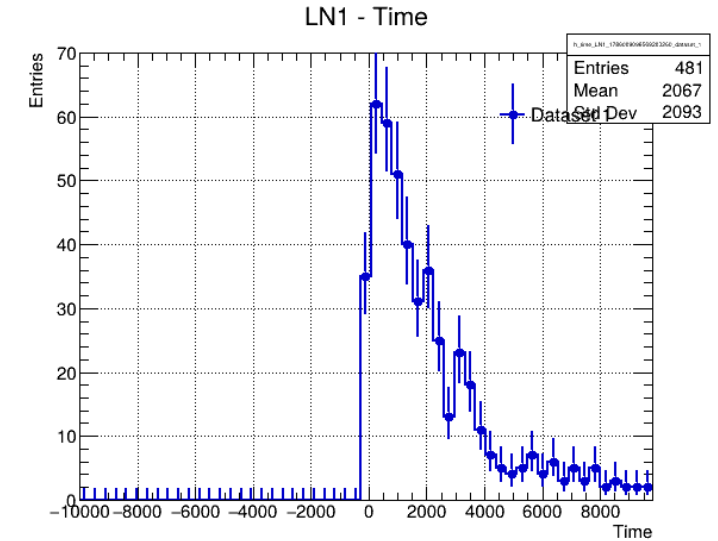
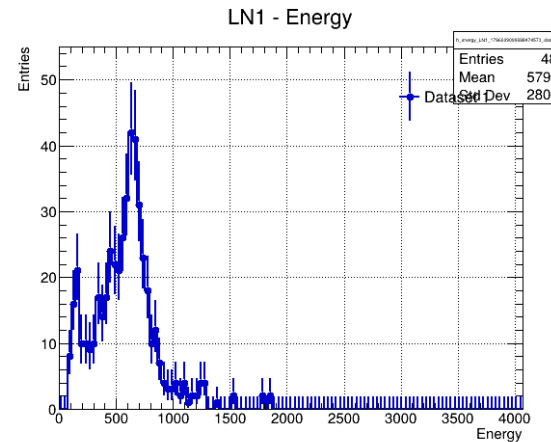
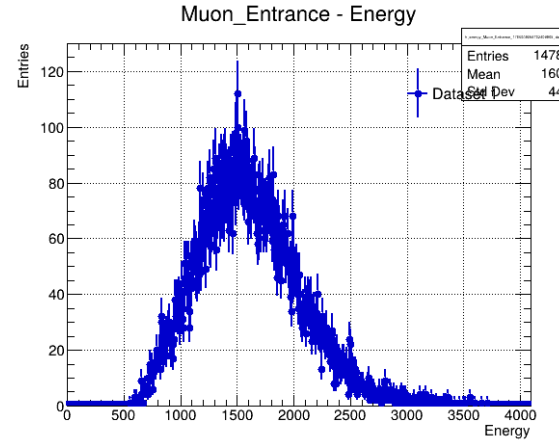


# Histogram Helper



## Main Responsibilities

- Parse 1D/2D quantity expressions
- Resolve histogram settings from configuration
- Create ROOT TH1/TH2 histograms with RDataFrame
- Support multiple datasets simultaneously
- Apply normalization, lifetime and background corrections

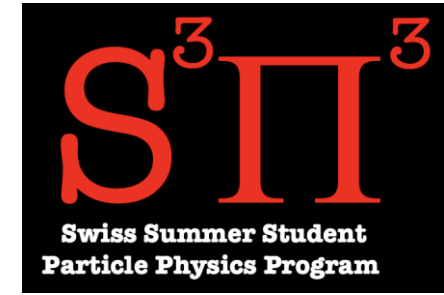


### Processing Flow



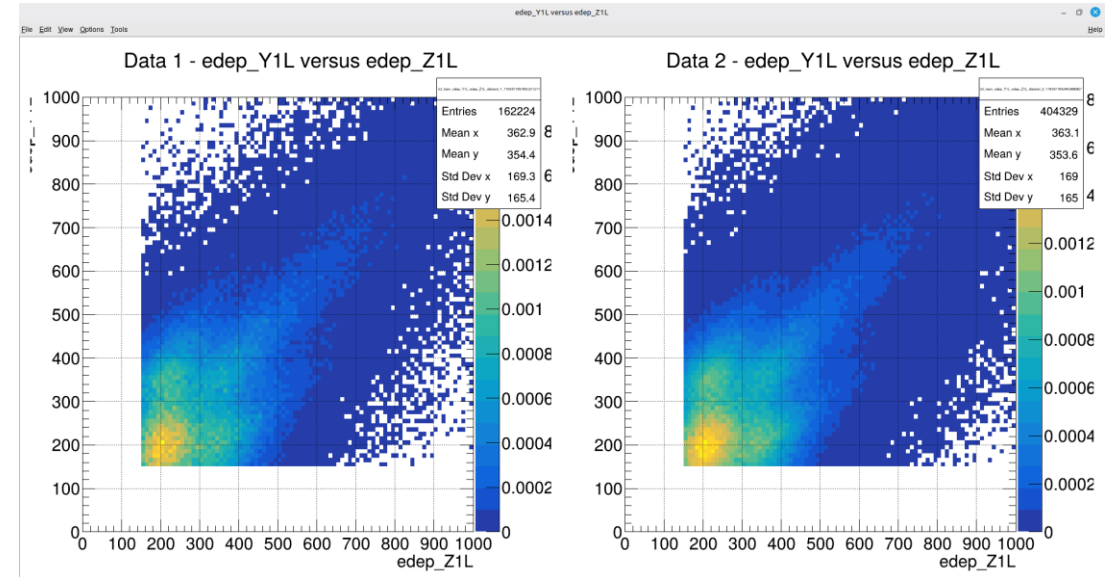


# Canvas Helper

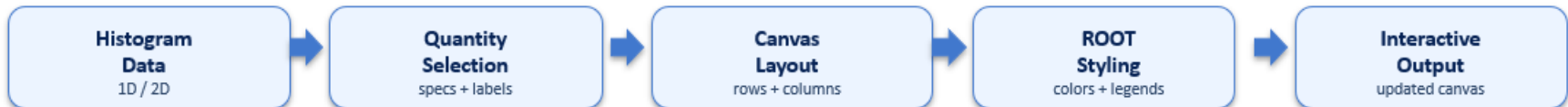


## Main Responsibilities

- Builds one ROOT TCanvas with one pad per selected quantity
- Handles colors, markers, grids, ticks and legends
- Supports normalized and non-normalized y-axis labels
- Creates separate canvases for each 2D expression
- Uses safe ROOT names for dynamically generated canvas objects

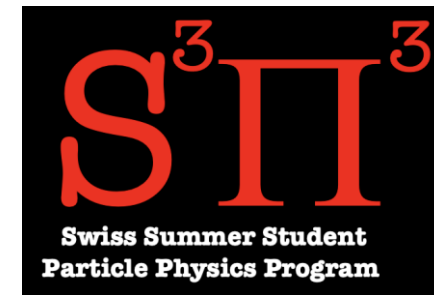


### Visualization Flow





# Coincidence Helper



## How the Coincidence Helpers Work

(The time is the average time of the first two detectors entered)

### 1 Input: Coincidence Detector Groups

User provides a list of detector groups.

```
[["LN3", "LF3"],
 ["LF1", "LF2", "LF3"]]
```

#### validate\_coincidence\_detector\_groups()

- ✓ Must be a list of lists
- ✓ Every detector must be a non-empty string
- ✓ No duplicates within a group
- ✓ Each group must have at least 2 detectors
- ✓ No duplicate groups
- ✓ At least one group required

#### Validated Output

```
[["LN3", "LF3"],
 ["LF1", "LF2", "LF3"]]
```

### 2 Build ROOT Filter for Each Group

\_build\_detector\_filter() creates the C++ filter used in ROOT to select the requested detectors.

#### Group 1: ["LN3", "LF3"]

(detector == "LN3" || detector == "LF3")

#### Group 2: ["LF1", "LF2", "LF3"]

(detector == "LF1" || detector == "LF2" || detector == "LF3")

Used in ROOT like:

```
tree->Draw("run:subrun:event:detector:time",
 (detector == "LN3" || detector == "LF3"))
```

### 3 Data from ROOT

ROOT returns rows for the selected detectors. Each row is one detector hit.

| Row | run  | subrun | event | detector | time (ns) |
|-----|------|--------|-------|----------|-----------|
| 0   | 1001 | 1      | 10    | LN3      | 1000.0    |
| 1   | 1001 | 1      | 10    | LF3      | 1200.0    |
| 2   | 1001 | 1      | 11    | LN3      | 2000.0    |
| 3   | 1001 | 1      | 11    | LF1      | 2050.0    |
| 4   | 1001 | 1      | 11    | LF3      | 2080.0    |
| 5   | 1001 | 1      | 12    | LF2      | 3000.0    |
| 6   | 1002 | 1      | 10    | LN3      | 100.0     |
| 7   | 1002 | 1      | 10    | LF3      | 150.0     |

### 4 Group Rows by (run, subrun, event)

\_group\_coincidence\_rows() organizes rows into events. For each event, it collects the times per detector.

```
{(1001, 1, 10): {
  "LN3": [1000.0],
  "LF3": [1200.0]
},
 (1001, 1, 11): {
  "LN3": [2000.0],
  "LF1": [2050.0],
  "LF3": [2080.0]
},
 (1001, 1, 12): {
  "LF2": [3000.0]
},
 (1002, 1, 10): {
  "LN3": [100.0],
  "LF3": [150.0]
}}
```

### 5 Find Coincidences Within Each Group

For each event and for each detector group, the code checks if all detectors have a hit and if their times are close enough (within the coincidence window).

#### Group 1: ["LN3", "LF3"] (window = 500 ns) → time = average(LN3, LF3)

| Event (run,subrun,event) | Times                       | Δt (ns) | Within 500 ns? | Average Time (ns)            |
|--------------------------|-----------------------------|---------|----------------|------------------------------|
| (1001, 1, 10)            | LN3: 1000.0, LF3: 1200.0    | 200.0   | ✓ Yes          | (1000.0 + 1200.0)/2 = 1100.0 |
| (1001, 1, 11)            | LN3: 2000.0, LF3: 2080.0    | 80.0    | ✓ Yes          | (2000.0 + 2080.0)/2 = 2040.0 |
| (1001, 1, 12)            | Only LF2 → missing detector | —       | ✗ No           | —                            |
| (1002, 1, 10)            | LN3: 100.0, LF3: 150.0      | 50.0    | ✓ Yes          | (100.0 + 150.0)/2 = 125.0    |

#### Group 2: ["LF1", "LF2", "LF3"] (window = 500 ns) → time = average(LF1, LF2)

| Event (run,subrun,event) | Times                            | All Detectors Present? | Within 500 ns? | Average Time (ns) |
|--------------------------|----------------------------------|------------------------|----------------|-------------------|
| (1001, 1, 10)            | Missing LF1 or LF2               | ✗ No                   | —              | —                 |
| (1001, 1, 11)            | LF1: 2050.0, LF2: —, LF3: 2080.0 | ✗ No (LF2 missing)     | —              | —                 |
| (1001, 1, 12)            | Missing LF1 or LF3               | ✗ No                   | —              | —                 |
| (1002, 1, 10)            | Missing LF1 or LF3               | ✗ No                   | —              | —                 |

### 6 Output: Coincidences

The function returns the events and the coincidence time (average of the first two detectors in the group) that satisfy each group.

#### Group 1: ["LN3", "LF3"] (time = average(LN3, LF3))

(1001, 1, 10) → t = 1100.0 ns (Δt = 200.0 ns)  
(1001, 1, 11) → t = 2040.0 ns (Δt = 80.0 ns)  
(1002, 1, 10) → t = 125.0 ns (Δt = 50.0 ns)

#### Group 2: ["LF1", "LF2", "LF3"] (time = average(LF1, LF2))

No coincidences found.

#### Legend

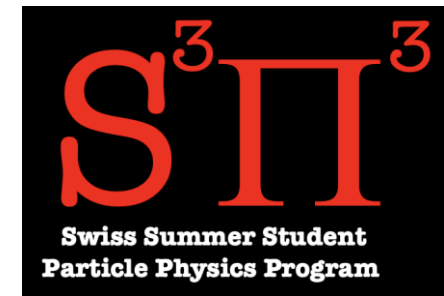
Δt = max(time) - min(time) within the group  
Window = coincidence time window (e.g., 500 ns)

- Filter requested detector combinations
- Group rows by run, subrun and event
- Require each detector exactly once
- Compute coincidence average time from detector pairs
- Automatically determine histogram ranges and labels





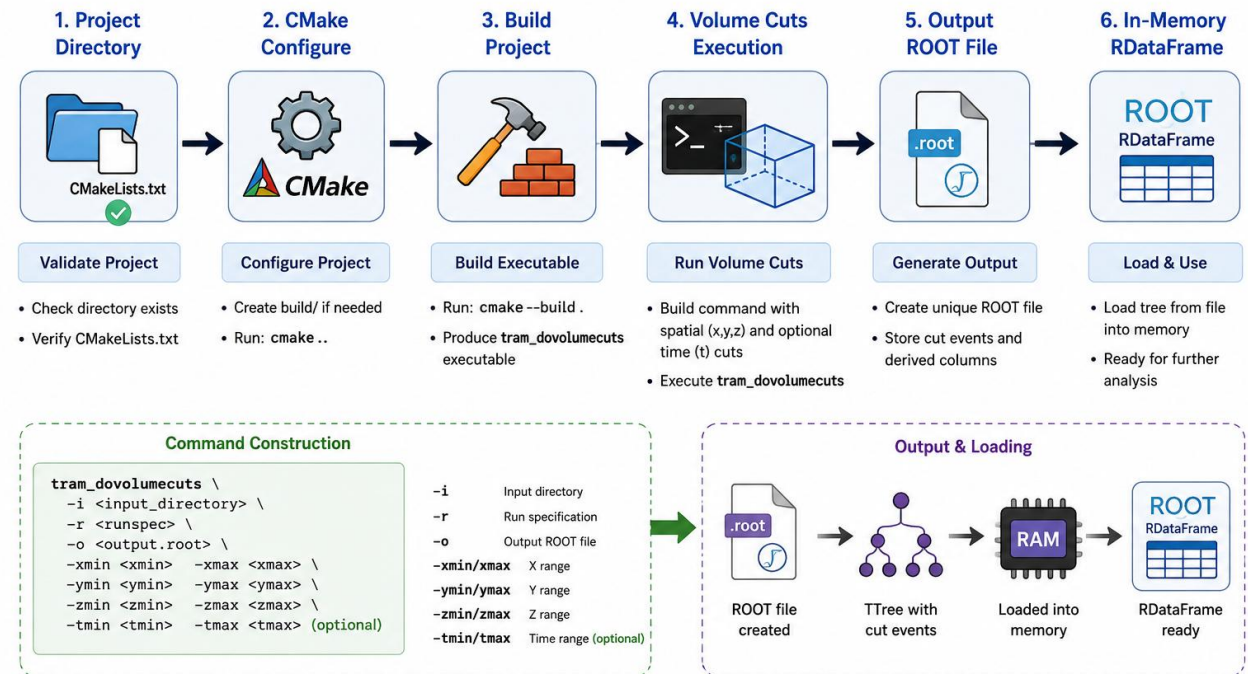
# TRAM Build Helper



## Key Responsibilities

- Validate TRAM project and CMake configuration
- Automatically configure and build the project
- Generate unique ROOT output files
- Execute volume-cut commands
- Load output TTrees directly into memory
- Clean up temporary files automatically

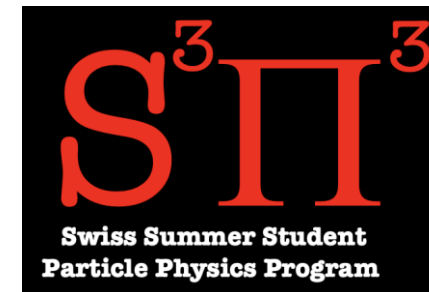
## TRAM Build Helper – Workflow



**Purpose:** Automate the full chain from building the TRAM tool to producing in-memory data for efficient analysis.



# Load Data to RDataFrame



## load\_data()

Return an RDataFrame for the specified runs.

### Parameters

The following parameters control how the data are located and loaded.

| Parameter              | Type              | Description                                             | Default                           |
|------------------------|-------------------|---------------------------------------------------------|-----------------------------------|
| runspec                | str               | Run specification (e.g., “run12345”, “run12345_1,2,3”). | (required)                        |
| local_directory        | str   Path   None | Path to local directory with ROOT files.                | None                              |
| host                   | str               | Remote host name.                                       | “lepp-c-3po”                      |
| user                   | str               | Remote user name.                                       | “leming”                          |
| remote_directory       | str               | Remote directory containing ROOT files.                 | “data0/leming/BT2025/kosmas/loaf” |
| tree_name              | str               | Name of the TTree inside the ROOT files.                | “t2_tram”                         |
| number_of_threads      | Optional[int]     | Number of threads for ROOT (None = use all available).  | None                              |
| add_run_subrun_columns | bool              | Add RunNumber and SubrunNumber columns.                 | False                             |

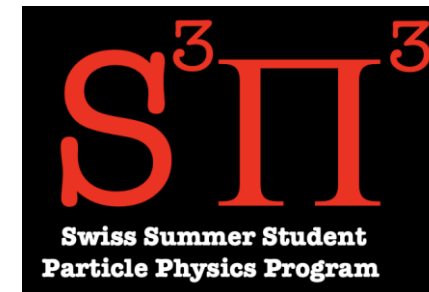
### Notes

- Either local\_directory or (host, user, remote\_directory) must be provided.





# Plot Detector Data from Loaf files



## plot\_detector\_data()

Plot detector data as 1D/2D histograms with optional filters, corrections, and coincidence selections.

### Parameters

The following parameters control how the data are processed and plotted.

| Parameter                    | Type                                                   | Default                        | Description                                                |
|------------------------------|--------------------------------------------------------|--------------------------------|------------------------------------------------------------|
| dataframe                    | Any   list[Any]                                        | —                              | RDataFrame or list of RDataFrames to plot.                 |
| quantities                   | list[str]   None                                       | None                           | Quantities to plot (e.g., "Y1L", "Y1L:Z1L").               |
| bins                         | int   list[int]   None   None                          | None                           | Number of bins per quantity (or list matching quantities). |
| ranges                       | tuple[float, float]   list[tuple[float, float]]   None | None                           | Axis ranges per quantity (or single range for all).        |
| detectors                    | list[str]   None                                       | None                           | Subset of detectors to include.                            |
| dataframe_labels             | list[str]   None                                       | None                           | Labels for datasets in legends.                            |
| cut_filter                   | str   list[str]   None                                 | None                           | RDataFrame filter expression(s) to apply.                  |
| normalized                   | bool                                                   | False                          | Normalize histograms to unit area.                         |
| lifetime_correction          | bool                                                   | False                          | Apply lifetime correction.                                 |
| apply_background_subtraction | bool                                                   | False                          | Subtract background if configured.                         |
| coincidence_detector_groups  | list[list[str]]   None                                 | None                           | Coincidence detector groups (e.g. [{"LN3", "LF3"}, ...]).  |
| coincidence_event_column     | str                                                    | "DEFAULT_EVENT_COLUMN"         | Event column for coincidence matching.                     |
| config_filename              | str   Path                                             | "detectors.json"               | Detector configuration file.                               |
| histogram_config_filename    | str   Path                                             | "load_histogram_defaults.json" | Histogram defaults configuration file.                     |
| output_filename              | str   Path   None                                      | None                           | Base name for saved outputs (optional).                    |
| output_formats               | str   list[str]                                        | "pdf"                          | Output format(s): e.g. "pdf", "png", "pdf.png".            |

### Returns

dict[str, Any]

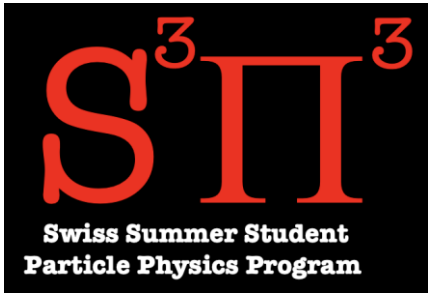
Dictionary containing canvases and metadata. Typical keys include:  
"canvases", "canvases\_2d", "legends", "drawn\_histograms", "output\_files"

### Notes

- Output files are written only if output\_filename is provided.



# Plot Data from Loaf & Tram files



## plot\_tram\_data()

Plot TRAM detector data as 1D/2D histograms with optional filters, corrections, and straight-coincidence.

### Parameters

| Parameter        | Type                              | Default    | Description                                 |
|------------------|-----------------------------------|------------|---------------------------------------------|
| dataframe        | Any   list[Any]                   | —          | RDataFrame or list of RDataFrames.          |
| quantities       | str   list[str]                   | (required) | Quantities to plot (e.g. "Energy", "dE:E"). |
| bins             | int   list[int]   None            | None       | Number of bins per quantity.                |
| ranges           | tuple   list   list[tuple]   None | None       | Axis ranges per quantity.                   |
| dataframe_labels | list[str]   None                  | None       | Labels for datasets in legends.             |
| cut_filter       | str   list[str]   None            | None       | RDataFrame filter expression(s).            |

### Options & Corrections

| Parameter                    | Type                           | Default | Description                                                     |
|------------------------------|--------------------------------|---------|-----------------------------------------------------------------|
| normalized                   | bool                           | False   | Normalize histograms to unit area.                              |
| lifetime_correction          | bool                           | False   | Apply lifetime correction.                                      |
| apply_background_subtraction | bool                           | False   | Subtract background if configured.                              |
| straight_coincidence         | dict(str, float   None)   None | None    | Compute straight-coincidence (e.g. {"tmin": ..., "tmax": ...}). |

### Config & Output

|                           |                   |                                |                                         |
|---------------------------|-------------------|--------------------------------|-----------------------------------------|
| histogram_config_filename | str   Path        | "tram_histogram_defaults.json" | Histogram defaults configuration file.  |
| output_filename           | str   Path   None | None                           | Base name for saved outputs (optional). |
| output_formats            | str   list[str]   | "pdf"                          | Output formats (e.g. "pdf", "png").     |

### Returns

dict[str, Any]

Dictionary with canvases, metadata, legends, and output file paths.

## plot\_loaf\_data()

Plot Loaf (TOF) detector data as 1D/2D histograms with optional filters and corrections.

### Parameters

| Parameter        | Type                              | Default    | Description                                 |
|------------------|-----------------------------------|------------|---------------------------------------------|
| dataframe        | Any   list[Any]                   | —          | RDataFrame or list of RDataFrames.          |
| quantities       | str   list[str]                   | (required) | Quantities to plot (e.g. "Y1L", "Y1L:Z1L"). |
| bins             | int   list[int]   None            | None       | Number of bins per quantity.                |
| ranges           | tuple   list   list[tuple]   None | None       | Axis ranges per quantity.                   |
| dataframe_labels | list[str]   None                  | None       | Labels for datasets in legends.             |
| cut_filter       | str   list[str]   None            | None       | RDataFrame filter expression(s).            |

### Options & Corrections

| Parameter                    | Type | Default | Description                        |
|------------------------------|------|---------|------------------------------------|
| normalized                   | bool | False   | Normalize histograms to unit area. |
| lifetime_correction          | bool | False   | Apply lifetime correction.         |
| apply_background_subtraction | bool | False   | Subtract background if configured. |

### Config & Output

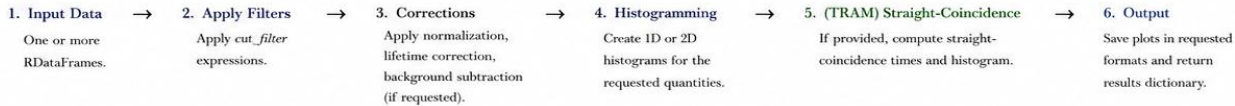
|                           |                   |                                |                                         |
|---------------------------|-------------------|--------------------------------|-----------------------------------------|
| histogram_config_filename | str   Path        | "loaf_histogram_defaults.json" | Histogram defaults configuration file.  |
| output_filename           | str   Path   None | None                           | Base name for saved outputs (optional). |
| output_formats            | str   list[str]   | "pdf"                          | Output formats (e.g. "pdf", "png").     |

### Returns

dict[str, Any]

Dictionary with canvases, metadata, legends, and output file paths.

### Common Workflow

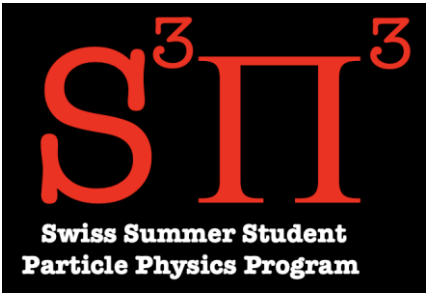


### Key Differences

- plot\_tram\_data(): includes optional straight\_coincidence computation.
- plot\_loaf\_data(): optimized for Loaf (TOF) quantities and defaults.



# Get Loaf Histograms & Plot Loaf Histograms



## LOAF Histogram Utilities

Utilities for creating and visualizing LOAF histograms from data.

### get\_loaf\_histograms()

Create histograms from LOAF data and return them in a dictionary (no plotting).

#### Parameters

| Parameter        | Type                             | Default    | Description                                      |
|------------------|----------------------------------|------------|--------------------------------------------------|
| dataframe        | Any   list[Any]                  | —          | RDataFrame or list of RDataFrames.               |
| quantities       | str   list[str]                  | (required) | Quantities to histogram (e.g. "Y1L", "Y1L:Z1L"). |
| bins             | int   list[int]   None   None    | None       | Number of bins per quantity.                     |
| ranges           | None   tuple   list per quantity | None       | Axis ranges per quantity.                        |
| dataframe_labels | list[str]   None                 | None       | Labels for datasets (for multi-DF cases).        |
| cut_filter       | str   list[str]   None           | None       | RDataFrame filter expression(s).                 |

#### Options & Corrections

| Parameter                    | Type | Default | Description                        |
|------------------------------|------|---------|------------------------------------|
| normalized                   | bool | False   | Normalize histograms to unit area. |
| lifetime_correction          | bool | False   | Apply lifetime correction.         |
| apply_background_subtraction | bool | False   | Subtract background if configured. |

#### Config

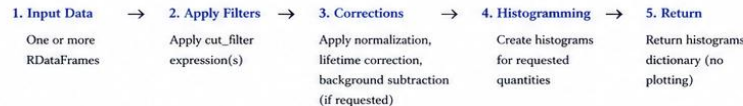
| Parameter                 | Type       | Default                        | Description                            |
|---------------------------|------------|--------------------------------|----------------------------------------|
| histogram_config_filename | str   Path | "loaf_histogram_defaults.json" | Histogram defaults configuration file. |

#### Returns

dict[str, dict[str, Any]]

Dictionary of histograms and metadata organized by quantity and dataframe label.

#### Workflow: get\_loaf\_histograms()



### plot\_loaf\_histograms()

Plot previously created LOAF histograms.

#### Parameters

| Parameter       | Type       | Default      | Description                                                           |
|-----------------|------------|--------------|-----------------------------------------------------------------------|
| histograms      | Any        | (required)   | Histogram dictionary returned by get_loaf_histograms().               |
| quantity        | str   None | None         | If provided, plot only this quantity. Otherwise, plot all quantities. |
| dataframe_label | str        | "dataframe1" | Which dataframe label to plot (from the histograms dict).             |

#### Returns

None

Displays the plot(s).

#### Workflow: plot\_loaf\_histograms()

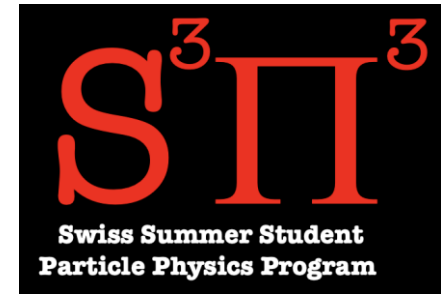


#### Notes:

Use get\_loaf\_histograms() to create histogram objects programmatically, and plot\_loaf\_histograms() to visualize them.

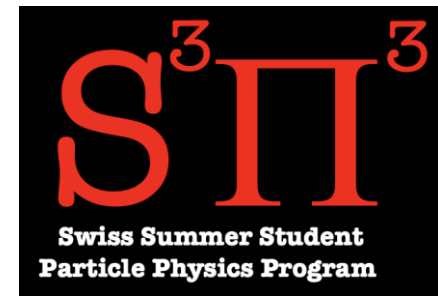


# Clear Results



| Feature                           | Benefit                                                      |
|-----------------------------------|--------------------------------------------------------------|
| Release ROOT objects and canvases | Frees memory after each analysis                             |
| Prevent memory leaks              | Keeps repeated analyses stable                               |
| Close temporary ROOT files        | Removes unused in-memory files                               |
| Disable implicit multithreading   | Useful when multithreading is not needed or causes conflicts |

✓ **Best Practice: Call `exit_detector_data()` at the end of every notebook or analysis script to ensure all ROOT resources are released before the next execution.**

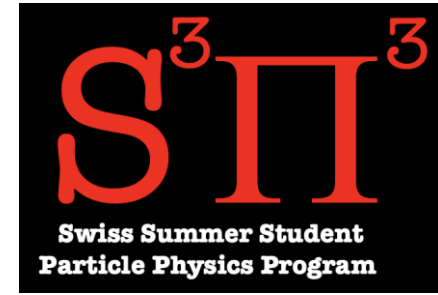


# Live Demonstration





# Future Developments



## 1) muSR Automatic Analysis

Automatically generate muSR plots when the required condition is detected.

## 2) Ranging Plots

## 3) Histogram Algebra

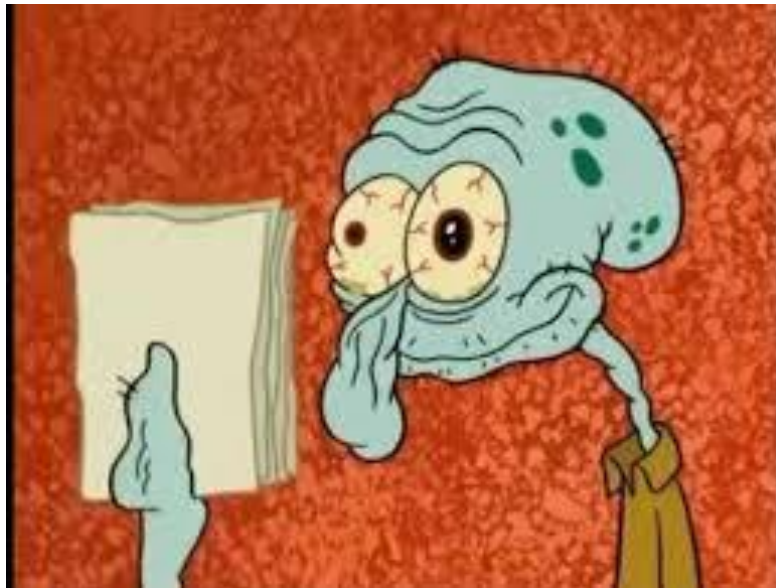
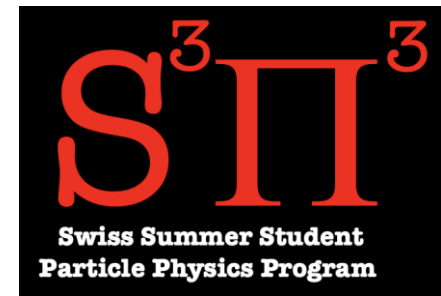
Direct addition, subtraction and arithmetic operations between histograms without manual extraction.

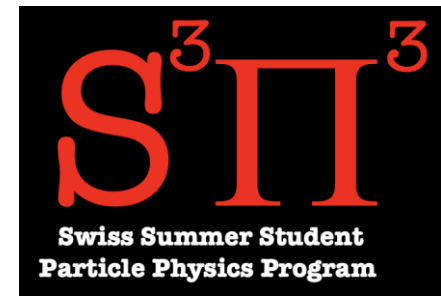
## 4) Live Monitoring Web Dashboard

Web server that automatically updates after each completed run and displays online plots (e.g. hitmaps, scintillators), potentially using the ROOT Web Server.



Thank you!!!

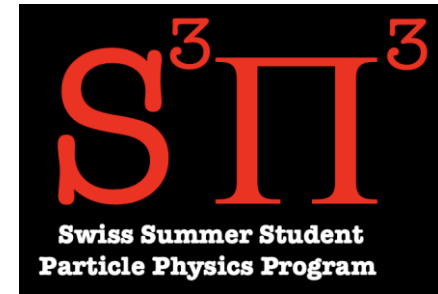




# Backup



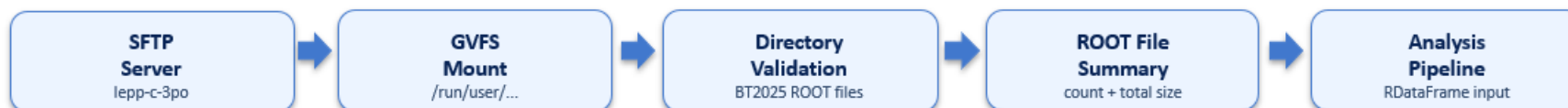
# Mount Helper



## Why Mounting Helpers?

- Automatically mounts the remote SFTP data location (or a specific local directory)
- Hides connection details from analysis notebooks
- Checks that the target ROOT-file directory exists
- Reports number and total size of available .root files
- Provides clear MountError / DataFrameError failures

## Workflow



## Main Responsibilities

- Builds the GVFS mount path from host and user
- Uses gio mount with password handling through pexpect
- Supports both remote mounted data and local directories
- Prints a reusable ROOT-directory summary
- Returns a validated local target directory for processing